

Universidad de Costa Rica

IF4101 - Lenguajes para Aplicaciones Comerciales

Proyecto 2: Desarrollo de una aplicación web para punto de venta

Gestión de cuentas bancarias y registro de órdenes de venta

Informe de avance técnico

Equipo 5

- Caleb Hernández Vega
- Sebastián Cordero Quesada
- Josué Delgado Corrales
- Alejandro Porras

Fecha de avance: 1 de julio de 2026

Tabla de contenidos

1. Sentencia del problema
2. Descripción general de la solución
3. Arquitectura lógica y estructura física del proyecto
4. Funcionalidades implementadas y verificadas
5. Base de datos
6. API RESTful desarrollada
7. Manejo transaccional
8. Frontend WPF
9. Accesibilidad digital
10. Evidencia de avance por archivos y commits
11. Pendientes críticos
12. Bitácora de avance verificado
13. Conclusiones de avance

1. Sentencia del problema

El Proyecto 2 solicita el desarrollo de una aplicación de punto de venta con backend en ASP.NET Core Web API y frontend en WPF. El sistema debe consumir servicios RESTful, utilizar SQL Server como motor de base de datos y aplicar una arquitectura por capas. Para el Equipo 5, el actualizador asignado corresponde a la gestión de cuentas bancarias.

Además del actualizador, el sistema debe registrar una orden de venta con patrón maestro-detalle. La orden debe permitir seleccionar cliente, agregar productos con cantidad, mostrar detalle, subtotal, impuesto y total, remover productos, incrementar o decrementar cantidades y actualizar inventario al procesar la venta.

Este informe resume únicamente el avance técnico verificado en el repositorio local y remoto del proyecto. No se atribuyen tareas no verificadas a integrantes que no tengan evidencia concreta en archivos, commits o funcionalidades revisadas.

2. Descripción general de la solución

La solución implementada se organiza como una aplicación separada físicamente en backend, frontend WPF y proyectos compartidos. La comunicación entre el WPF y el backend se realiza mediante HTTP hacia la API REST.

Área	Descripción
Backend	Expone endpoints REST para catálogos, cuentas bancarias y órdenes.
Frontend WPF	Permite gestionar cuentas bancarias y registrar órdenes de venta.
Base de datos	SQL Server con tablas para bancos, compañías, clientes, empleados, productos, cuentas bancarias, órdenes y detalles.
Proyectos compartidos	Contienen contratos, DTOs, entidades y excepciones usadas por backend y WPF.
Regla de negocio principal	Cálculo de IVA y actualización de inventario durante el registro de la orden.

3. Arquitectura lógica y estructura física del proyecto

La solución mantiene una arquitectura por capas:

Capa	Proyecto	Responsabilidad
Presentación backend	Proyecto_backend/SalesPro.Api	Controladores REST, Swagger y configuración de API.
Negocio	Proyecto_backend/SalesPro.Business	Validaciones, reglas de negocio y coordinación de operaciones.
Datos	Proyecto_backend/SalesPro.Data	Repositorios ADO.NET, consultas SQL y manejo de transacciones.
Presentación WPF	Proyecto_WPF/SalesPro.Wpf	Vistas, ViewModels, comandos y consumo de API.
Dominio	Proyecto_compartido/SalesPro.Domain	Entidades y excepciones del dominio.
Contratos	Proyecto_compartido/SalesPro.Contracts	DTOs, requests y responses compartidos.

La estructura física actual es:

```
Proyecto_backend/  
  SalesPro.Api  
  SalesPro.Business  
  SalesPro.Data  
  database  
  scripts  
  
Proyecto_WPF/  
  SalesPro.Wpf  
  
Proyecto_compartido/  
  SalesPro.Domain  
  SalesPro.Contracts
```

Esta separación se realizó para facilitar los entregables solicitados: Proyecto_backend comprimido y Proyecto_WPF comprimido. El directorio Proyecto_compartido contiene clases requeridas por ambos proyectos.

4. Funcionalidades implementadas y verificadas

4.1 Gestión de cuentas bancarias

Se encuentra implementado el CRUD de cuentas bancarias en backend y WPF.

Operaciones disponibles:

- listar cuentas bancarias;
- buscar cuentas bancarias;
- obtener cuenta bancaria por identificador;
- crear cuenta bancaria;
- actualizar cuenta bancaria;
- eliminar cuenta bancaria.

Archivos principales:

- Proyecto_backend/SalesPro.Api/Controllers/CuentasBancariasController.cs

- Proyecto_backend/SalesPro.Business/Services/CuentaBancariaService.cs
- Proyecto_backend/SalesPro.Data/Repositories/CuentaBancariaRepository.cs
- Proyecto_WPF/SalesPro.Wpf/Views/CuentasBancariasView.xaml
- Proyecto_WPF/SalesPro.Wpf/ViewModels/CuentasBancariasViewModel.cs

4.2 Registro de orden de venta

Se encuentra implementado el flujo base de registro de orden:

- búsqueda y selección de cliente mediante ventana WPF;
- búsqueda y selección de productos mediante ventana WPF;
- ingreso de cantidad;
- detalle maestro-detalle en tabla;
- incremento y decremento de cantidades;
- remoción de productos;
- cálculo visual de subtotal, IVA estimado y total estimado;
- envío de la orden al backend;
- procesamiento transaccional de la orden en la capa de datos.

Archivos principales:

- Proyecto_backend/SalesPro.Api/Controllers/OrdenesController.cs
- Proyecto_backend/SalesPro.Business/Services/OrdenService.cs
- Proyecto_backend/SalesPro.Data/Repositories/OrdenRepository.cs
- Proyecto_WPF/SalesPro.Wpf/Views/NuevaOrdenView.xaml
- Proyecto_WPF/SalesPro.Wpf/Views/BuscarClienteWindow.xaml
- Proyecto_WPF/SalesPro.Wpf/Views/BuscarProductoWindow.xaml
- Proyecto_WPF/SalesPro.Wpf/ViewModels/NuevaOrdenViewModel.cs

5. Base de datos

La base de datos se crea mediante el script:

Proyecto_backend/database/00_create_salespro.sql

El script crea la base SalesPro y las tablas principales:

Tabla	Propósito
ParametroSistema	Parámetros generales como IVA.
Banco	Bancos registrados.
Compania	Compañías registradas.
Cliente	Clientes que pueden asociarse a órdenes.
Empleado	Empleados que registran órdenes.
Producto	Productos vendibles, precios e inventario.
Compania_Cuenta_Bancaria	Cuentas bancarias asignadas a compañía y banco.

Tabla	Propósito
Pos_Orden	Encabezado de orden de venta.
Pos_Orden_Detalle	Detalle de productos por orden.

El script incluye datos semilla para bancos, compañía, clientes, empleado, productos, cuentas bancarias y parámetro IVA.

Validación ejecutada:

```
powershell -ExecutionPolicy Bypass -File .\Proyecto_backend\scripts\setup-localdb.ps1
```

Resultado verificado:

```
Bancos: 3
Productos: 4
IVA: 13
```

6. API RESTful desarrollada

La API expone recursos para catálogos, cuentas bancarias y órdenes.

Recurso	URL	Método	Descripción
Bancos	/api/catalogos/bancos	GET	Lista bancos registrados.
Compañías	/api/catalogos/companias	GET	Lista compañías registradas.
Clientes	/api/catalogos/clientes?buscar={texto}	GET	Busca clientes por texto.
Productos	/api/catalogos/productos?buscar={texto}	GET	Busca productos por texto.
Cuentas bancarias	/api/cuentas-bancarias	GET	Lista o busca cuentas bancarias.
Cuentas bancarias	/api/cuentas-bancarias/{id}	GET	Obtiene una cuenta bancaria por id.
Cuentas bancarias	/api/cuentas-bancarias	POST	Crea una cuenta bancaria.
Cuentas bancarias	/api/cuentas-bancarias/{id}	PUT	Actualiza una cuenta bancaria.
Cuentas bancarias	/api/cuentas-bancarias/{id}	DELETE	Elimina una cuenta bancaria.
Órdenes	/api/ordenes	POST	Crea una orden de venta.
Órdenes	/api/ordenes/{numeroOrden}	GET	Obtiene una orden por número.

La API también cuenta con Swagger para revisión interactiva:

```
http://localhost:5294/swagger
```

Las pruebas manuales están en:

```
Proyecto_backend/SalesPro.Api/SalesPro.Api.http
```

7. Manejo transaccional

La transacción principal se implementa en:

El flujo transaccional realiza:

1. apertura de conexión SQL;
2. inicio de `SqlTransaction`;
3. validación de cliente;
4. validación de empleado, si aplica;
5. validación de productos;
6. inserción de encabezado de orden;
7. inserción de detalles;
8. descuento de inventario;
9. `Commit` si todo finaliza correctamente;
10. `Rollback` si ocurre una excepción antes del commit.

Este punto es crítico porque en la evaluación anterior del proyecto pasado se evidenció una debilidad al explicar transacciones. En este proyecto, la transacción está localizada y se puede defender desde una clase concreta.

8. Frontend WPF

La aplicación WPF contiene dos funcionalidades principales:

Vista	Archivo	Función
Gestión de cuentas bancarias	CuentasBancariasView.xaml	CRUD visual de cuentas bancarias.
Nueva orden	NuevaOrdenView.xaml	Registro de orden maestro-detalle.
Búsqueda de cliente	BuscarClientewindow.xaml	Selección de cliente asociado a la orden.
Búsqueda de producto	BuscarProductoWindow.xaml	Selección de producto y cantidad.

El WPF consume la API mediante:

Proyecto_WPF/SalesPro.Wpf/Services/ApiClientService.cs

La vista de orden ya no depende de escribir manualmente el identificador del cliente. Actualmente se abre una ventana de búsqueda, se consulta el endpoint de clientes y se asigna el cliente seleccionado al `ViewModel`.

9. Accesibilidad digital

Se incorporaron ajustes básicos de accesibilidad digital en WPF relacionados con criterios de la Ley 7600:

- nombres accesibles en controles principales mediante `AutomationProperties.Name`;
- ayudas contextuales mediante `AutomationProperties.HelpText`;
- `ToolTip` en controles importantes;
- orden de tabulación para navegación por teclado;
- mensajes de estado con `AutomationProperties.LiveSetting`;
- encabezados de pantalla con `AutomationProperties.HeadingLevel`.

Estos ajustes se implementaron en las vistas principales, pero no sustituyen una validación formal con lector de pantalla o usuario final.

10. Evidencia de avance por archivos y commits

Commits relevantes verificados

Commit	Descripción
cebaee4	Estabilización documental y organización del proyecto.
e79a330	Agregado de accesibilidad básica en WPF.
6186da3	Búsqueda y selección de cliente en la orden.
eed8d21	Separación física entre backend, WPF y proyectos compartidos.

Validaciones ejecutadas

Compilación:

```
dotnet build SalesPro.slnx --no-restore
```

Resultado:

```
0 errores, 0 advertencias
```

Base de datos:

```
powershell -ExecutionPolicy Bypass -File .\Proyecto_backend\scripts\setup-localdb.ps1
```

Resultado:

```
Base montada correctamente en LocalDB.
```

11. Pendientes críticos

Los siguientes puntos no deben marcarse como concluidos hasta tener evidencia:

Pendiente	Motivo
Pruebas completas del archivo .http	Se requiere confirmar todos los endpoints y casos negativos.
Prueba manual completa del CRUD de cuentas bancarias en WPF	Debe validarse creación, edición, búsqueda y eliminación desde interfaz.
Prueba manual completa de orden desde WPF	Debe validarse cliente, productos, cantidades, total e inventario.
Evidencia de rollback por stock insuficiente	Debe documentarse antes/después de inventario y ausencia de orden parcial.
Documento final formal	Este archivo es informe de avance, no entrega final.
Diagramas ER y dominio finales	Deben hacerse con nombres claros, multiplicidades y relaciones no ambiguas.

12. Bitácora de avance verificado

Esta bitácora registra únicamente avance verificado en el repositorio. No se atribuyen tareas a integrantes sin evidencia concreta.

Fecha	Responsable verificado	Actividad	Evidencia
2026-06-29	Sebastián Cordero	Base inicial del proyecto por capas.	Commit 6b2ffbc.
2026-06-30	Sebastián Cordero	Organización documental y estructura de trabajo.	Commit cebae4.
2026-06-30	Sebastián Cordero	Accesibilidad básica en WPF.	Commit e79a330.
2026-06-30	Sebastián Cordero	Búsqueda y selección de cliente para orden.	Commit 6186da3.
2026-06-30	Sebastián Cordero	Separación física de backend, WPF y compartido.	Commit eed8d21.

Nota: este informe no certifica aportes de otros integrantes porque no se revisó evidencia individual suficiente en esta etapa.

13. Conclusiones de avance

El proyecto cuenta con una base funcional y defendible para el Equipo 5. La arquitectura por capas está implementada, la base de datos puede montarse localmente, el backend compila, el WPF compila y ya existe una separación física alineada con los entregables solicitados.

El avance más importante frente a los puntos débiles del proyecto anterior es que el manejo transaccional está localizado en una clase concreta y puede explicarse desde código. También se corrigió un riesgo importante de rúbrica al implementar búsqueda real de cliente para la orden.

Sin embargo, todavía no se debe considerar el proyecto finalizado. Las prioridades restantes son completar pruebas .ht tp, ejecutar pruebas manuales completas del WPF, documentar evidencia de transacción y elaborar los diagramas finales con claridad suficiente para evitar observaciones de ambigüedad.